

Kandidatnummer: 34

Kandidatnummer: 16

Kandidatnummer: 92

Emnekode: PGR208
Emnenavn: Android Programmering
Innleveringsdato: 2.12.2025
Antall ord: 1795
Gruppe eksamen | Rapport del



Høyskolen Kristiania

Høsten 2025

Gruppenummer: 17

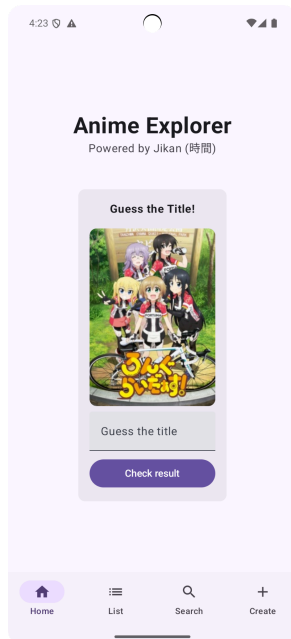
«Denne besvarelsen er gjennomført som en del av utdannelsen ved Høyskolen Kristiania.
Høyskolen er ikke ansvarlig for oppgavens metoder, resultater, konklusjoner eller
anbefalinger.»

App-dokumentasjon

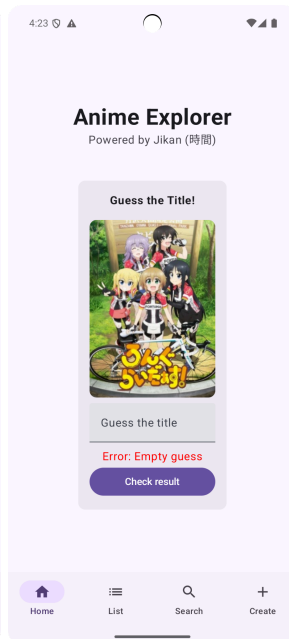
Del 1. Redegjørelse av app per skjerm

Skjerm - HomeScreen

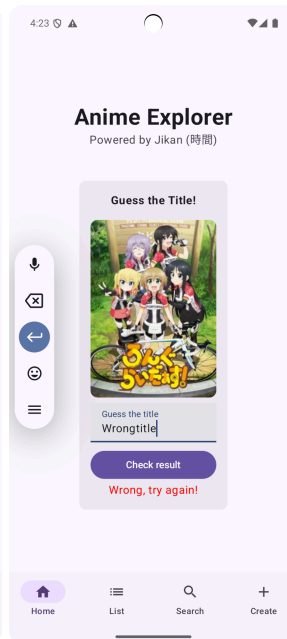
Bilde 1



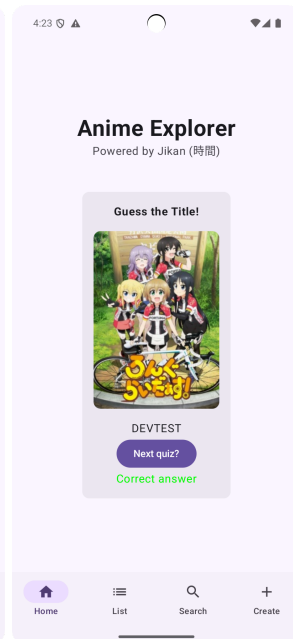
Bilde 2



Bilde 3



Bilde 4



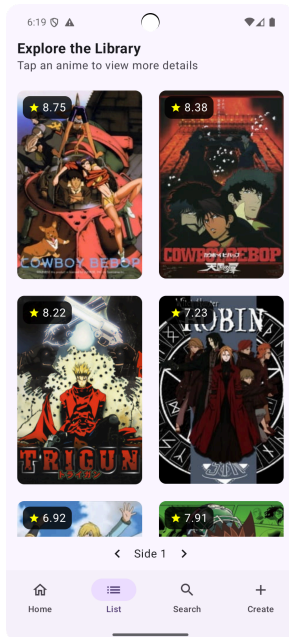
Denne skjermen er knyttet til oppgave “Skjerm #4: Egendefinert skjerm”. Denne skjermen fungerer som startsidene til applikasjonen vår. Den består av en tittel med undertittel og for at siden ikke skal være static så valgte vi å legge til en morsom “quiz funksjon” hvor brukeren får et bilde og skal gjette tittelen på den animen.

Vi startet med å utforske hvordan quiz-delen skulle fungere. I tidlig prototyping fase så brukte vi listOf med noen Int som vi visste var id'er i API'et. Den hentet så et random tall som parameter til getAnimeById(). Senere fant vi ut at Jikan API'et hadde en egen spørring som het “getRandomAnime” (Jikan API, u.å.), det gjorde ViewModel vesentlig mindre komplisert så vi endret det. Etter vi hadde gjort det så fokuserte vi på feilmendinger til brukeren, se bilde 2 og 3.

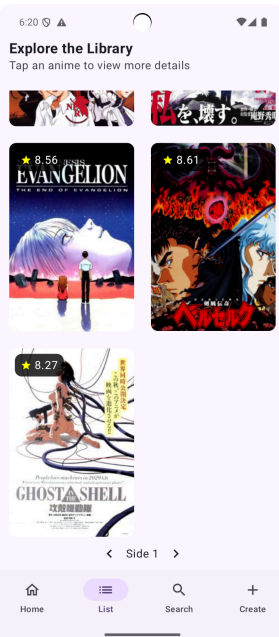
Det kompliserte med denne screenen var å holde styr på isCorrect variabelen. Denne brukes veldig mange steder i Screen for å finne ut av hva som skal vises. F.eks. på bilde 4 så ser man at Check result knappen har blitt byttet ut med en ny knapp.

Skjerm - AnimeListScreen

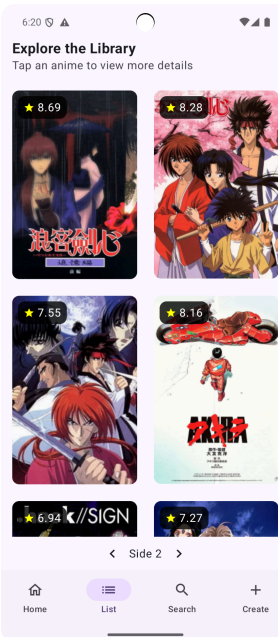
Bilde 5



Bilde 6



Bilde 7



Denne skjermen er knyttet til oppgave “Skjerm #1: Vise alle animer fra JIKAN anime”. Vår ekstra funksjon på denne siden er at vi har tatt i bruk pagination fra API’et som gjør det mulig å bla gjennom hele biblioteket til Jikan API’et, istedenfor å kun hente page 1 “default page”.

Under utviklingsfasen av denne skjermen så startet vi med å ha en vertikal liste av artikler under hverandre, men vi ble ikke fornøyd med utseendet. Vi lette etter inspirasjon og fant ut at Jikan.moe hadde en showcase på deres nettside (Jikan, u.å.). For at vi skulle få til et grid-design som vi fant i showcase, måtte vi nok en gang gå gjennom dokumentasjonen. Der fant vi noe som heter LazyVerticalGrid, dette passet til vår sammenheng (Google, 2025b).

Rent teknisk så er screenen koblet opp mot en ViewModel som har 3 funksjoner. En `getAllAnimes()` med parameter for sidetall som kobles til JikanRepository og 2 funksjoner for sidehåndtering. Vi vurderte om vi skulle ha de to funksjonene `nextPage()` og `previousPage()` i Screen, men med tanke på mvvm-arkitektur så vi det som mer hensiktsmessig å tildele ViewModel denne logikken.

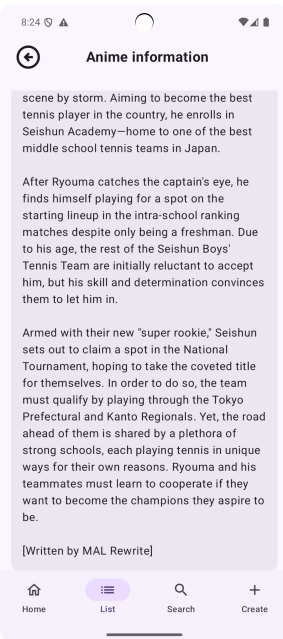
En utfordring vi møtte på underveis var at griden ikke tok hensyn til elementene under den, så pagination-delen under som viser sidetall og page navigation, ble ikke synlig. For å løse dette satte vi først en fast høyde, men den ville ikke ha tilpasset seg bra med ulike skjermstørrelser. Vi husket å ha lest noe om fraksjon i en av slidene fra forelesningen, så vi søkte det opp og leste oss igjen opp i dokumentasjonen. Løsningen ble å sette `weight(1f)` på LazyVerticalGrid. `weight(1f)` gjør at den tar 100% av den tilgjengelige restplassen, noe som vil si at den tar hensyn til de andre komponentene (Google, 2025a).

Skjerm - AnimeDetailsScreen

Bilde 8



Bilde 9



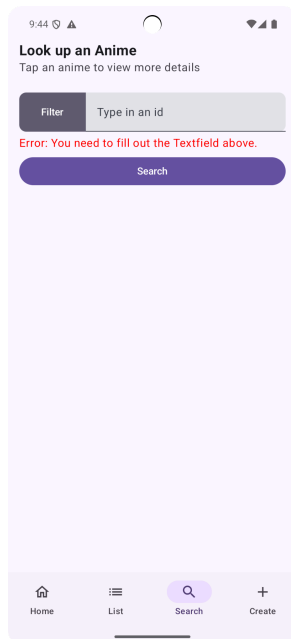
Denne skjermen er en detailscreen som er koblet til AnimeListScreen. Den eneste måten å navigere seg til denne skjermen er ved at brukerne trykker på en AnimeListItem. På denne siden så får man et forstørret bilde, med tittel og beskrivelse “synopsis” av animeen.

I likhet med AnimeListScreen er denne skjermen også koblet opp mot sitt eget item “AnimeItem” under components mappen. I AnimeItem har vi også hatt fokus på at når man kaller på et API, så vet man aldri hva slags resultat man vil få, eller om man i det hele tatt får noe. Derfor har vi null-håndtering for tittel og synopsis, samt et placeholder-bilde hvis det mot formodning oppstår en feil under innhenting av bildet. På Bilde 8 og 9 ser man hvordan den scroller.

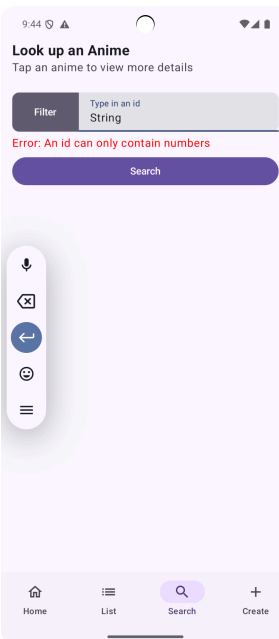
En utfordring vi møtte på her var stylingen for å få Text-tittelen i midten av skjermen, øverst, på linje med tilbakeknappen. Vi kom fram til en litt kreativ løsning på dette ved å sette 1 fraksjon på maxwidth, med en 40px padding på end for å ta høyde for IconButton, som er 40px. Dette gjorde at tittelen ble plassert perfekt i midten av skjermen øverst.

Skjerm - AnimeSearchScreen

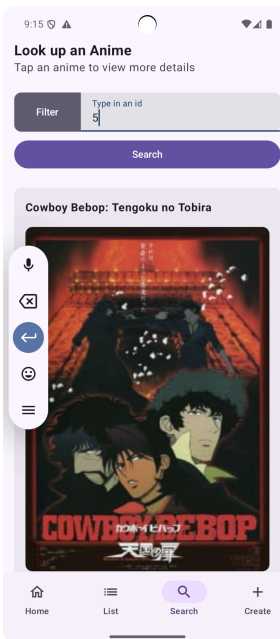
Bilde 10



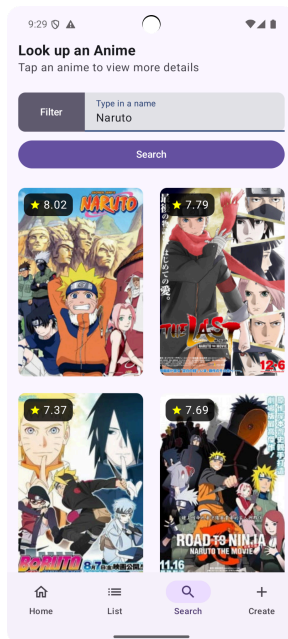
Bilde 11



Bilde 12



Bilde 13



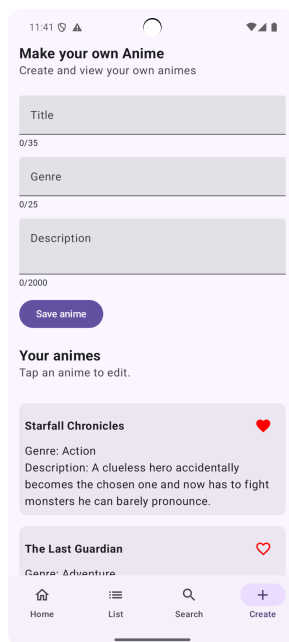
Denne skjermen er knyttet til oppgave “Skjerm #2: Søke etter en anime fra JIKAN anime” Vår tilleggsfunksjon på denne skjermen er muligheten for å søke etter en anime med navn (title).

Vi startet tidlig i prosessen med å finne ut av hvordan vi skulle gjøre det mulig å søke på både name og id. Vår første tanke var å ha to tekstfelt, men vi ble ikke fornøyd med den tilnærmingen. Vi valgte i stedet å ha en filter knapp som gjør at man kan veksle mellom å søke etter id eller title. Her måtte vi nok en gang søke oss opp i dokumentasjonen for å lære oss DropdownMenu (Google, 2025c).

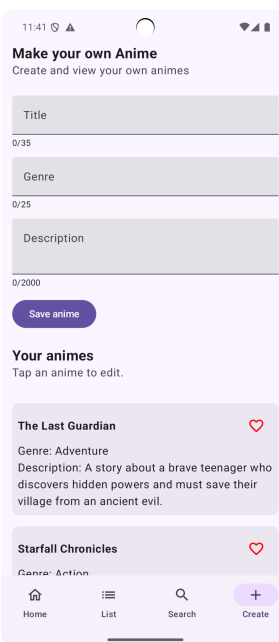
Denne skjermen håndterer to ulike API-kall i ViewModel, både searchById og searchByName. Forenklet forklart så bruker vi en variabel kalt searchMode i screen som holder på hva brukerne filterer etter, også har vi mange enkle if-sjekker som oppdaterer både errorMessage og verdien til searchMode for å kunne vite hvilken tekst som skal inn i dropdown, input-label også videre. Vi har også hatt mye fokus på feilmeldinger på denne delen (se bilde 10 - 11) hvor vi blant annet sjekker om feltet er tomt etc.

For å holde styr på hva som skal vises til brukeren av resultatet, bruker vi idResult.value?.let til å vise en anime siden ID er unik, og .isNotEmpty() til å vise en liste når man søker på navn. Funksjonene searchById og searchByName i ViewModell nullstiller hverandre basert på hvilken som blir kalt. Dette gjør at det alltid blir vist riktig i screen (se bilde 12 - 13).

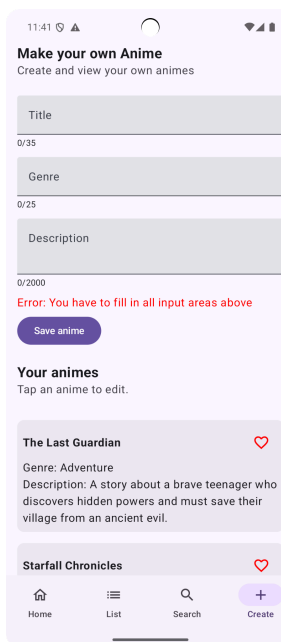
Bilde 14



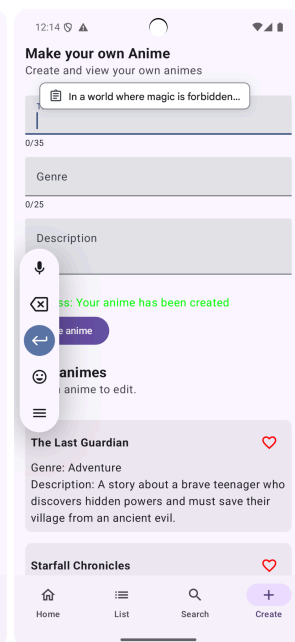
Bilde 15



Bilde 16



Bilde 17



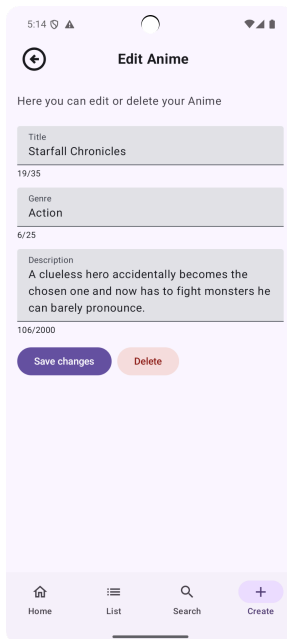
Denne skjermen er knyttet til oppgave “Skjerm #3: "Brukerens egne animeideer”. Vi valgte å splitte denne oppgaven i to forskjellige screens hvor man på denne screenen kun har mulighet til å legge til en anime, og se anime-listen fra databasen. Når man trykker på en `AnimeCreateItem` så vil man komme til redigering screenen. Tilleggsfunksjonen for denne siden er å kunne favorisere sine egne animes.

Skjermen er koblet til `AnimeCreateViewModel`, som inneholder tre funksjoner: `setAnimes()` henter og setter listen over animer som ligger i databasen. `insertAnime()` legger til nye anime-objekter som brukeren oppretter. `setFavoriteStatus()` oppdaterer favorittstatusen (boolean) når brukeren trykker på hjerteikonet. Anime som har `favorite = true` vises øverst i kolonnen.

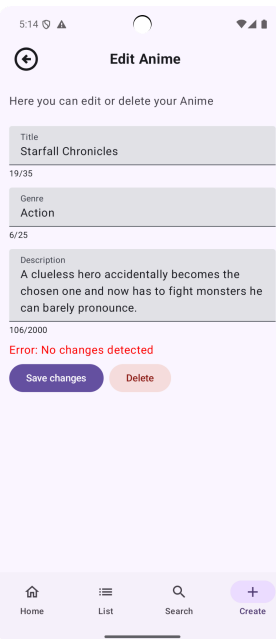
En utfordring vi møtte på denne skjermen var under debug-testingen hvor vi fant ut at hvis man fylte ut en lang tekst i beskrivelse-tekstfeltet så ble “Save anime”-knappen ikke lenger synlig. Dette er fordi en vanlig `Column` ikke har noen scroll som default i motsetning til f.eks. en `LazyColumn`. Vi løste dette ved å samle tekstfeltene og knappen i en `Column` med en `VerticalScrollModifier`.

Skjerm - AnimeEditScreen

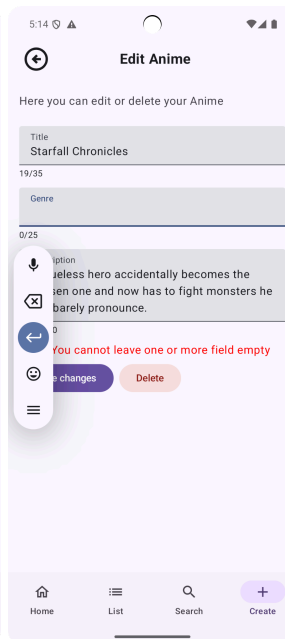
Bilde 18



Bilde 19



Bilde 20



Denne skjermen er knyttet til oppgave “Skjerm #3: Brukerens egne animeideer”. Dette er en detailscreen som man kommer til når man trykker på en `AnimeCreateItem` i `AnimeCreateScreen`. Her får man muligheten til å redigere de forskjellige verdiene for et item, samt muligheten for å slette.

På denne siden hadde vi feilhåndtering og database spørringer i fokus. Dersom verdiene i alle tekstboksene skulle være lik seg selv, så vil brukeren få en feilmelding (se bilde 19), selv om det fint hadde funket å ikke hatt den sjekken så er det en unødvendig database spørring som vi valgte å stoppe. Vi har også en sjekk som krever at alle feltene er fylt ut (se bilde 20).

Under utviklingen oppdaget vi at anime ble satt til null når brukeren trykket på Delete knappen. Dette førte til at vår tidligere feilhåndtering viste en melding som indikerte at den ikke fant anime. Tidligere hadde vi hele koden wrappet i en `anime?.let` med en elvis-operator for feilmelding. Denne metoden gikk vi altså bort fra, og erstattet det ved å opprette en `isDeleted` variabel og en ny sjekk som sjekker om både `anime == null` og om `isDeleted` er ikke er true. Så la vi til under delete-knappen at den setter `isDeleted` til true når den trykkes på. På denne måten vil feilmeldingen kun vises når den skal.

Del 2. Redegjørelse av planlegging og utvikling

Vi startet arbeidet med å møte fysisk på skolen for å planlegge en mulig løsning for oppgaven. Siden vi var 3 personer i gruppen måtte vi finne ut av hvilke tilleggsfunksjoner som var hensiktsmessige, og drøfte potensielle løsninger for hver oppgave. Her tok vi i bruk FigmaJam for å lage en wireframe for å få en oversikt over arbeidsmengden og hvilke filer vi trengte. Denne fremgangsmåten gjorde at vi enkelt fikk oversikt over hvilke deler som tilhørte hva og vi fikk blant annet diskutert gjenbruk av items, og kommet ganske fort inn på det overordnede tekniske.

Når vi hadde en ide om omfanget til arbeidet begynte vi å diskutere arbeidsfordeling. Vi diskuterte våre kunnskaper om Android programmering for å finne hverandres sterke og svake sider. Vi endte med å gi et hovedansvar for API-delen av appen mens de andre jobbet med databasedelen. Vi hadde likevel tett dialog under hele prosessen og vi gjennomgikk hverandres kode ofte for å sikre at kodestrukturen ikke varierte for mye.

Alle større avgjørelser som endring av mappestruktur, nye teknikker, spesielt de som var utenfor pensum som (dropdown og LazyVerticalGrid) diskuterte vi sammen slik at alle var enige og forsto hensikten bak koden.

Samarbeidet fungerte meget bra, vi hadde kontinuerlig samtaler og møter digitalt underveis i prosessen for å oppdatere hverandre på koden, og for å diskutere ulike teknikker. Det var noe kronglete siden vi ikke alle hadde kjennskap til GitHub funksjoner som pull request, version control osv. Så vi endte opp med å sette hovedansvar på en person for å sørge for å ha den oppdaterte filen. Dette var jo en litt treig og ueffektiv metode, spesielt når kanskje en fikset litt på en fil mens den andre designet det ble vanskelig å se hvilke endringer som ble gjort, men samlet sett så klarte vi å jobbe sammen på en god måte.

Litteraturliste

- Google. (2025a, 21. november). Compose modifiers. Android Developers.
<https://developer.android.com/develop/ui/compose/modifiers>
- Google. (2025b, 21. november). Lists and grids. Android Developers.
<https://developer.android.com/develop/ui/compose/lists>
- Google. (2025c, 21. november). Menus. Android Developers.
<https://developer.android.com/develop/ui/compose/components/menu>
- Jikan. (u.å.). Showcase. Hentet 1. desember 2025 fra <https://jikan.moe/showcase>
- Jikan API. (u.å.). getRandomAnime.
<https://docs.api.jikan.moe/#/random/getrandomanime>